Práctica 1 — Monte Carlo Tree Search sobre Conecta 4

Asignatura: Inteligencia Artificial Avanzada (IA4) **Profesor:** Dr. Luis Peña Sánchez **Curso:** 2026/2027 **Se publica:** lunes 28 de septiembre de 2026 **Entrega:** lunes 23 de noviembre de 2026, antes de clase **Peso:** 10 % de la nota final

---

1. Objetivos

Al terminar esta práctica seréis capaces de:

1. Implementar **MCTS** completo desde cero: selección con **UCT**, expansión, simulación y retropropagación.
2. Explicar **por qué** MCTS no necesita función de evaluación, y **medirlo** en vez de creerlo.
3. Diseñar un **experimento reproducible** con semilla fija, salida a CSV y un número de partidas suficiente para que las conclusiones se sostengan.
4. Decidir con criterio **cuándo usar MCTS y cuándo usar alfa-beta**, que es la pregunta que se pregunta en el examen.

2. De dónde viene esto

En la **sesión 2** vimos los tres sitios por donde se rompe minimax, y el peor de los tres era que **hace falta una función de evaluación y casi nunca la hay**. En la hoja de ejercicios de aquel día diseñasteis una función de evaluación para el **4 en raya**.

Pues bien: el rival de esta práctica es **exactamente ese minimax, con esa función de evaluación**. Y lo que tenéis que programar es un algoritmo que **no usa ninguna**.

La práctica contesta a dos preguntas, y la segunda es la que importa:

> ***1.*** *¿Puede MCTS competir con un alfa-beta que sí tiene una buena heurística?* ***2.*** *Y cuando al rival* ***se le quita la heurística****, ¿le sirve de algo buscar más hondo?*

3. El proyecto base

Descargad `MctsConecta4_base.zip` del Campus. Es un proyecto de consola en **C# (.NET 8)** que compila y arranca tal cual:

```
dotnet build
dotnet run
```

Está todo hecho menos el algoritmo. En concreto **ya funcionan**:

Fichero	Qué es
Conecta4.cs	El juego: tablero de 7 × 6, movimientos legales, detección de cuatro en raya

Fichero	Qué es
IJuego.cs	Lo único que MCTS necesita saber de un juego. Fijaos: **no hay función de evaluación**
Nodo.cs	Un nodo del árbol: visitas, recompensa, hijos y movimientos pendientes
Agentes.cs	AgenteAleatorio, AgenteMinimax (con heurística y **ciego**) y AgenteMcts
Torneo.cs	Enfrenta dos agentes N partidas **alternando quién empieza**, y exporta a CSV
Program.cs	Los modos de ejecución, incluido referencia, que genera la entrega

⚠ **Sólo se toca** `MotorMcts.cs`**.** Si modificáis el juego, los agentes o el torneo, los resultados dejan de ser comparables con los de referencia y con los de vuestros compañeros.

4. Tareas obligatorias (hasta 7 puntos)

4.1 Implementación — 4 puntos

`MotorMcts.cs` tiene **siete** `TODO`. Los seis primeros son el algoritmo; el séptimo es el presupuesto de tiempo.

TODO	Método	Puntos	Se trabaja en
1	Seleccionar — bajar por el árbol mientras el nodo esté expandido del todo	0,5	Taller I, 5 oct
2	MejorHijoUct — la fórmula de UCT	**1,0**	Taller I, 5 oct
3	Expandir — crear **un** hijo nuevo	0,5	Taller I, 5 oct
4	Simular — jugar al azar hasta el final	0,5	Taller I, 5 oct
5	Retropropagar — subir el resultado **invirtiendo el signo**	**1,0**	Taller I, 5 oct
6	MejorMovimiento — devolver la jugada **más visitada**	0,5	Taller I, 5 oct
7	El corte por tiempo dentro del bucle	se evalúa en **E4**	Taller II, 26 oct

Los TODO 2 y 5 valen el doble porque son los dos que se hacen mal con más frecuencia y los dos que no dan ningún error cuando están mal. El TODO 7 no puntúa aparte porque

sin él no hay experimento **E4**: se evalúa allí.

Antes de seguir, comprobad las tres pruebas de humo que están en el `README.md` del proyecto. **No puntúan: son un requisito.** Si no pasan, hay un fallo en la implementación y los números de los experimentos no valen:

1. `dotnet run -- traza` elige la **columna 3** en los seis presupuestos, y el porcentaje de visitas de esa columna **sube** con el presupuesto.
2. `dotnet run -- duelo 40 2000 4 42` da a MCTS **más del 45 %**.
3. Contra `AgenteAleatorio` con 500 iteraciones o más, MCTS gana **prácticamente siempre**.

4.2 Los cuatro experimentos — 3 puntos

Se lanzan todos de una vez con:

```
dotnet run -- referencia 40 42
```

que deja un `referencia_s42.csv`. **40 partidas por punto y semilla 42**, que es lo que hace comparables los resultados de todos los grupos.

Exp.	Qué se varía	Qué contesta	Puntos
E1	El presupuesto: 50, 100, 200, 500, 1.000, 2.000 y 5.000 iteraciones, contra Aleatorio y contra Minimax-4	¿Cuántas simulaciones hacen falta para que esto sirva de algo?	0,75
E2	El rival: Minimax **con** heurística y **ciego**, a profundidad 2, 4 y 6	**La pregunta del bloque**	**1,0**
E3	La constante **C**: 0 · 0,5 · 1 · 1,41 · 2 · 4	¿Qué pasa en los dos extremos?	0,75
E4	Presupuesto en **milisegundos**: 5, 10, 25, 50 y 100	El comportamiento *anytime*, que es el del juego real. **Ojo: depende de vuestra máquina.** Necesita el TODO 7	0,5

Los números por sí solos no puntúan: puntúa **lo que se dice sobre ellos** en el informe.

5. Tareas opcionales (hasta 3 puntos)

Se cogen las que se quiera. **No hace falta hacerlas todas** y no se puede pasar de 10.

Opción	Qué	Puntos
O1	**Reutilizar el árbol** entre turnos: al mover, quedarse con el subárbol del hijo elegido en vez de tirarlo. Contad **cuántas visitas heredáis** por jugada y medid **si** eso sube la tasa de victorias **y con cuánta confianza**	1,0
O2	***Rollout* con política**: en la simulación, si hay un movimiento que gana ya, jugarlo; y si el rival amenaza cerrar, taparlo. Medid el efecto	1,0
O3	**Monte Carlo plano**: implementadlo (N simulaciones por jugada legal, sin árbol) y comparadlo con MCTS a igual presupuesto	0,5
O4	**Gráficas** de E1, E3 y E4 hechas desde vuestro CSV, con la herramienta que queráis	0,5

⚠ **O2 tiene truco y por eso vale 1 punto**: un *rollout* con política es más lento, así que a igualdad de **milisegundos** hace menos simulaciones. Si sólo medís a igualdad de iteraciones, la comparación no vale. Medidlo de las dos maneras.

⚠ **Y O1 tiene otro, del que se habla poco**: lo que reutilizar regala es **un poco más de presupuesto**, y una mejora pequeña **puede no verse** con 40 partidas. Se puntúa **la medición, no el signo del resultado**: «la diferencia es de 2 puntos y el intervalo es de ±8, así que no puedo distinguirla de cero» es una respuesta **completa** y vale el punto entero. Inventarse una mejora, no.

6. Valores de referencia

Con la implementación de referencia, `dotnet run -- referencia 40 42` da esto:

E1 · presupuesto (% de puntuación de MCTS; victoria 1, tablas ½)

Iteraciones	vs Aleatorio	vs Minimax-4
50	97,5 %	2,5 %
100	100 %	2,5 %
200	100 %	15,0 %
500	100 %	40,0 %
1.000	100 %	48,8 %

Iteraciones	vs Aleatorio	vs Minimax-4
2.000	100 %	56,3 %
5.000	100 %	61,3 %

E2 · con heurística y sin ella (MCTS-2.000)

Profundidad del rival	**Minimax** con **heurística**	**Minimax** ciego
2	58,8 %	96,3 %
4	56,3 %	92,5 %
6	23,8 %	92,5 %

E3 · la constante C (MCTS-2.000 contra Minimax-4)

C	0	0,5	1	1,41	2	4
%	5,0	70,0	57,5	56,3	61,3	40,0

E4 · presupuesto de tiempo — y aquí no hay tabla de referencia

E4 se mide contra el reloj, así que sus cifras dependen de vuestra máquina y de lo cargada que esté. No es un defecto del enunciado: es la diferencia entre un presupuesto de iteraciones y un presupuesto de tiempo, que es el que tiene un juego de verdad.

Lo comprobamos: ejecutando la implementación de referencia **dos veces seguidas en el mismo ordenador**, E1, E2 y E3 dieron **exactamente los mismos números las dos veces** —26 filas de 26— y **E4 cambió hasta 17 puntos** en alguna fila.

Por eso el programa **calibra primero** —y lo podéis lanzar suelto con `dotnet run -- calibrar 42`, que además avisa si falta el TODO 7— y os dice cuántas iteraciones caben de verdad en vuestra máquina en cada presupuesto. En la de referencia:

ms por jugada	5	10	25	50	100
Iteraciones que caben	≈ 1.100	≈ 2.100	≈ 4.800	≈ 9.400	≈ 18.000

Y con esos presupuestos, contra `Minimax-4`, **tres ejecuciones de la misma orden en la misma máquina** dieron esto:

ms por jugada	5	10	25	50	100
Rango observado	55-68 %	54-69 %	67-76 %	67-75 %	93-99 %

Rangos, no cifras: eso es lo que se puede afirmar de un experimento medido contra el reloj.

Esa calibración va en la ficha, y es lo que hace que vuestro E4 se pueda interpretar.

O1 y O2 · qué esperar de las opcionales

O1 — reutilizar el árbol. Con la implementación de referencia, el subárbol que se hereda vale del orden de **64 visitas por jugada con 500 iteraciones**, **340 con 2.000** y **10.000 con un presupuesto de 8 ms**. Ése es el número que demuestra que lo habéis implementado bien. En cambio la tasa de victorias, medida con **1.000 partidas**, **no se movió**: 35,0 → 35,6 · 57,4 → 56,9 · 62,8 → 63,2, todo dentro de ±3 puntos. No es un fallo vuestro: heredar un 17 % del presupuesto vale poco, porque **doblarlo entero** —de 2.000 a 4.000 iteraciones— sólo sube 1,3 puntos.

O2 — *rollout* con política. A igualdad de **iteraciones** la mejora es grande y se ve con pocas partidas. A igualdad de **milisegundos** sigue siendo favorable, pero menos, y con presupuestos de 1 ms desaparece. Si os sale al revés, mirad si vuestra comprobación de «¿gana con esta columna?» está **copiando el tablero**: eso la hace más del doble de lenta.

Cómo se usan estas tablas

- **E1, E2 y E3 son deterministas**: con la misma semilla y la misma implementación salen siempre los mismos números. Pero **vuestra** implementación consumirá los aleatorios en otro orden, así que no vais a obtener estas cifras exactas, y eso es normal.
- Con 40 partidas, el error típico de un porcentaje cercano al 50 % es de unos **8 puntos**, así que una diferencia de hasta **15 puntos** no significa nada.
- Lo que **sí** tiene que coincidir es la **forma**: E1 creciente, E2 con la columna del ciego muy por encima de la otra, E3 hundido en los dos extremos y E4 creciente. Si algo de eso no os sale, hay un fallo, y el enunciado os acaba de decir dónde mirar.

7. Entregables

Un único **ZIP** llamado `P1_MCTS_Apellido1_Apellido2.zip` con:

1. **El proyecto completo**, que compile con `dotnet build` **sin tocar nada**. Sin `bin/`, sin `obj/` y sin la carpeta `.vs/`.
2. `referencia_s42.csv`, tal cual lo escribe el programa.
3. **La ficha de resultados** (`Ficha_Resultados_P1`), rellena, en PDF.
4. **El informe**, en PDF, de **dos páginas como máximo**.

Qué se espera del informe

No es un resumen de la teoría. Son **cuatro apartados cortos**, uno por experimento, y cada uno contesta a una pregunta concreta:

- **E1** — ¿A partir de cuántas simulaciones MCTS deja de ser un generador de números aleatorios? ¿Y por qué la curva se aplana?
- **E2** — ¿Le sirve al rival ciego pasar de profundidad 4 a profundidad 6? **Contestad con vuestros dos números.** ¿Qué dice eso sobre para qué sirve buscar más hondo?

- **E3** — ¿Qué le pasa a MCTS con C = 0? ¿Y con C = 4? Nombrad los dos fallos.
- **E4** — Con un presupuesto de **16 ms** —un *frame* a 60 fps—, ¿pondríais este MCTS en un juego? Justificadlo con **vuestra** calibración: ¿cuántas iteraciones caben en 16 ms en vuestra máquina, y qué porcentaje daba ese presupuesto en E1?

Y una conclusión de cuatro líneas: **¿cuándo usaríais MCTS y cuándo alfa-beta?**

8. Rúbrica

Apartado	Puntos
TODO 1, 3, 4, 6 (selección, expansión, simulación y jugada final)	2,0
TODO 2 — UCT correcto, con las visitas del hijo en el denominador	1,0
TODO 5 — retropropagación con el signo invertido	1,0
E1, E3 y E4 ejecutados y comentados (E4 incluye el TODO 7, el corte por tiempo)	1,5
E2 ejecutado y bien interpretado	**1,0**
Ficha completa, con la semilla y el CSV que la respalda	0,5
Opcionales O1-O4	hasta 3,0
Total	**10,0**

Lo que resta puntos

- Un proyecto que **no compile**: la práctica no se corrige. Se avisa y se recupera con la penalización de entrega fuera de plazo.
- Números en la ficha que **no salgan del CSV entregado**.
- Haber tocado `Conecta4.cs`, `Agentes.cs` o `Torneo.cs` sin decirlo.
- Conclusiones que **no se sostengan** sobre vuestros propios datos. Es preferible escribir «me sale al revés que en la referencia y no sé por qué» que adornar un resultado.

9. Normas

- **En parejas o individual.** No se admiten grupos de tres.
- **Entrega**: por el Campus, hasta el **lunes 23 de noviembre a las 18:00**.
- **Fuera de plazo**: −1 punto por cada día natural, hasta 5 días. Después, no se corrige.
- Se puede usar **cualquier ayuda**, incluida la de una IA, con una condición: tenéis que poder **explicar cada línea** de lo que entregáis. Se pregunta en clase.
- Y hay dos sesiones enteras de taller para hacerla con ayuda: **5 y 26 de octubre**. Quien viene a las dos sale con la mitad hecha y comprobada.